CoMem: Persistent Intermediate-Residual Memory for Bounded-Read Long-Context Inference

Anonymous ACL submission

Abstract

Retrieval bounds the online token set but recomputes every model layer, whereas state reuse avoids computation by retaining layer-wise caches. We introduce **CoMem**, a cross-query memory interface that stores one intermediate residual per context token, selects a bounded set of chunks for each query, and continues computation only through the upper transformer layers. The split depth exposes an explicit quality–latency–storage trade-off: on a matched selected pack, resuming at layer 12 yields a $1.403\times$ Read-phase speedup over raw-text replay while reducing a 15-cell RULER macro by 3.12 points. A rank-32 self-distillation adapter recovers most of the otherwise severe interface loss without updating the backbone. For repeated-query workloads, the one-time Write is amortized after roughly 9 queries at 32k and 26–28 queries at 128k in our serving measurements. The combined bounded-selection operating point keeps online prefill near 1.1 s and 18.7 GB at 128k, versus 71.4 s and 50.0 GB for dense prefill on the same H20; this $64.9\times$ figure includes the benefit of selecting a fixed working set and is not a depth-only effect. On a paired multikey diagnostic, controlled interface ablations identify missing lower-layer document context during independent chunk writing as a major source of fidelity loss. A deployable overlap-Write variant raises that diagnostic from 92.5 to 98.5–99.0 while leaving persistent storage and Read cost unchanged, at the price of additional one-time Write compute. CoMem therefore provides a measured computation-reuse frontier for long-context serving, rather than uniform quality dominance over raw-text retrieval.

1 Introduction

Long-context language models face two different costs. Processing a long prompt increases time to first token, while retaining its layer-wise key–value (KV) states increases accelerator memory (Vaswani et al., 2017; Kwon et al., 2023). Both costs recur when a document, conversation, or codebase answers many queries.

Existing methods usually reduce the *token* axis. Text retrieval selects a small raw-text prompt and then recomputes the full network (Lewis et al., 2020; Xu et al., 2024); KV compression retains fewer full-depth states (Xiao et al., 2024; Zhang et al., 2023; Li et al., 2024; Cai et al., 2024; Liu et al., 2024a); and chunk-cache systems reuse request states when their contextual dependencies permit it (Agarwal et al., 2025). These approaches provide valuable operating points, but they do not jointly offer independently writable cross-query memory objects, bounded query-conditioned selection, and direct continuation from an intermediate model depth.

We study that interface with **CoMem**. During WRITE, context chunks run through layers $[0{:}j)$ and store their residuals. During SELECT, a retriever chooses a fixed number of chunks. During READ, their states are packed with the query and only layers $[j{:}L)$ execute with full cross-chunk attention (Figure 1). The split j is a reuse knob: a deeper split prepays more work and stores the same one-vector-per-token representation, but reduces per-query upper-layer computation. The limiting case $j{=}0$ is a matched raw-text replay baseline with the same retrieved chunks, order, sink, and mask.

This framing separates two effects that are easily conflated. *Bounded selection* removes most online tokens; *depth reuse* skips lower-layer recomputation on the selected tokens. The former produces the large full-context speedups. The latter is smaller but measurable: with the same LoRA, pack, and examples, $j{=}12$ gives a $1.403\times$ Read-phase speedup over $j{=}0$ at a 3.12-point RULER cost. Its one-time Write becomes worthwhile only under reuse—about 9 repeated queries at 32k and

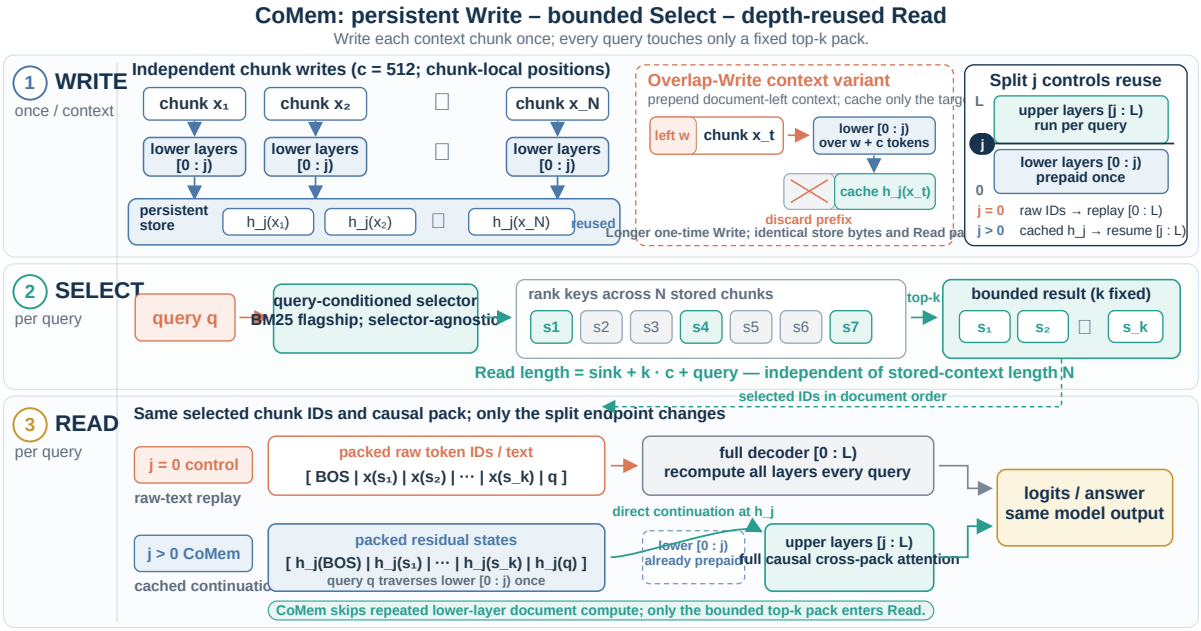

Figure 1: **CoMem treats transformer depth as a cross-query reuse axis.** WRITE stores an intermediate residual for each chunk, SELECT retrieves a fixed working set, and READ resumes the upper layers with the query present. The matched $j{=}0$ path retrieves the same raw-text chunks but recomputes every layer. A small left-overlap during Write restores lower-layer document context without increasing persistent storage or per-query Read work.

26–28 at 128k in the measured serving cohort. At equal GPU-resident online latency, raw-text replay remains stronger on our mixed diagnostic cohort, so CoMem should be read as a quality–latency–storage frontier rather than an accuracy replacement for RAG.

What limits fidelity? An intermediate residual can be linearly informative without being natively compatible with the remaining layers. A lightweight self-distillation LoRA repairs much of this interface mismatch. On a paired multi-key diagnostic, controlled Write variants show that missing lower-layer document context is a major source of the remaining deployable gap; position remapping alone does not repair it. A document-contextual control matches full replay on that diagnostic, and a deployable 32-token overlap improves 92.5 to 98.5 while adding only a small theoretical increment to lower-layer Write FLOPs. This result turns the fidelity analysis into an actionable design principle: context belongs primarily on the Write side.

Contributions.

- **A cross-query intermediate-residual memory interface** combining independent writes, bounded selection, and direct upper-layer continuation. Its depth split makes computation reuse explicit rather than hiding it inside a token-cache policy.
- **A matched accounting of the frontier.** We separate bounded selection from depth-only reuse, report persistent bytes, one-time Write, per-query Read, external-store costs, and measured repeated-query crossover points.
- **A controlled interface diagnosis and repair.** Same-pack replay, same-depth adaptation, a continuous-state oracle, and a context/position factorization identify lower-layer Write context as a major source of error on the paired multi-key diagnostic; a small overlap-Write recovers most of that observed gap without enlarging the store or Read working set.

2 Related Work

We position CoMem by the object that is stored, the scope over which it can be reused, and the computation performed after retrieval. Table 1 summarizes representative interfaces; it is not an empirical ranking.

Token-axis KV reduction. StreamingLLM retains a recent window and attention sinks (Xiao

Method family	Stored object	Cross-query persistence	Independent writes	Bounded query selection	Direct upper-layer continuation
Text RAG (Lewis et al., 2020)	token IDs	✓	✓	✓	–
Token-axis compression (Li et al., 2024; Cai et al., 2024)	full-depth KV	–	–	✓	–
HCache (Gao et al., 2025)	intermediate activation	context-bound	–	–	✓
KV-Direct (Qasim et al., 2026)	residual	request-bound	–	–	KV reconstruction
Cache-Craft (Agarwal et al., 2025)	chunk KV	✓	context-sensitive	✓	partial KV repair
REFORM (Song et al., 2025)	compressed KV + tokens	request-bound	streaming	✓	KV recomputation
MemoryLLM (Wang et al., 2024)	latent pool	✓	–	fixed pool	–
CoMem	depth-j residual	✓	✓	✓	✓

Table 1: Interface-level comparison, not an empirical ranking. HCache is a checkpoint/replay precedent. Cache-Craft repairs context-dependent chunk caches, and REFORM gathers tokens before recomputing KV. CoMem combines persistent independent residual writes, bounded query-conditioned selection, and direct upper-layer continuation.

et al., 2024); H2O and SnapKV evict positions (Zhang et al., 2023; Li et al., 2024); PyramidKV, ZigZagKV, LCKV, and MiniCache redistribute, condense, or merge layer-wise states (Cai et al., 2024; Zhong et al., 2025; Wu and Tu, 2024; Liu et al., 2024a). Query-aware systems such as Quest, RetrievalAttention, ACRE, FIER, A^{2}ATS, HeteroCache, and REAL dynamically retrieve or retain relevant KV (Tang et al., 2024; Liu et al., 2024b; Qian et al., 2025; Wang et al., 2025; He et al., 2025; Shi et al., 2026; Li et al., 2026). These methods primarily optimize which *tokens or KV entries* remain available. CoMem instead stores one intermediate residual per token and reconstructs the selected tokens' upper-layer computation. This depth partition is complementary to quantization, eviction, and heterogeneous storage.

Intermediate activations and recomputation. HCache checkpoints intermediate activations to restore evicted execution state (Gao et al., 2025). KV-Direct reconstructs layer-wise KV from residuals but keeps the full token sequence (Qasim et al., 2026). ILRe uses intermediate-layer representations to retrieve tokens for context compression (Liang et al., 2025). REFORM incrementally compresses context, gathers salient tokens, and recomputes their KV (Song et al., 2025). These works make intermediate states a useful systems object. CoMem differs in the conjunction of *cross-query persistence*, *independently written chunks*, *bounded query-time selection*, and *direct continuation from the corresponding depth*. We restrict our novelty claim to that interface combination.

Reusable context and chunk caches. CacheBlend, RAGCache, IMDC, and Cache-Craft reuse or repair cached states for retrieval-augmented serving (Yao et al., 2024; Jin et al., 2024; Zeng et al., 2024; Agarwal et al., 2025). Cache-Craft is particularly close in workload: retrieved chunks recur across queries, but arbitrary context changes make their KV caches only partially reusable. It selectively recomputes cache regions to restore quality. CoMem stores a single residual tensor rather than layer-wise chunk KV and exposes split depth as an explicit reuse knob; our overlap-Write variant analogously shows that limited neighboring context can repair independently cached chunks.

Learned and architectural memory. MemoryLLM maintains a fixed-size latent pool, LongMem retrieves external states through a side network, and RecursiveSummarizing stores natural-language summaries (Wang et al., 2024, 2023b,a). YOCO and CEPE redesign the model to share or encode context efficiently (Sun et al., 2024; Yen et al., 2024); Activation Beacon, KV-CAT, LLoCO, and IndexMem train compressible or compensating memory representations (Zhang et al., 2024; Gelberg et al., 2026; Tan et al., 2024; Yang et al., 2026). CoMem keeps the backbone frozen and trains only an optional interface LoRA. Its residual store grows linearly with the written corpus and is therefore not a constant-capacity memory.

Raw-text retrieval. Conventional RAG retrieves text and recomputes the reader (Lewis et al., 2020; Xu et al., 2024). Our $j{=}0$ baseline is not intended to represent every RAG system: it fixes CoMem's selector, selected chunk order, sink, and attention mask, making it the direct endpoint of the depth axis. Dense-retriever experiments in the appendix show that realistic retrieval can introduce a separate recall bottleneck. The central comparison in this paper is therefore not "memory versus RAG" in general, but raw-text full-depth replay versus residual upper-layer

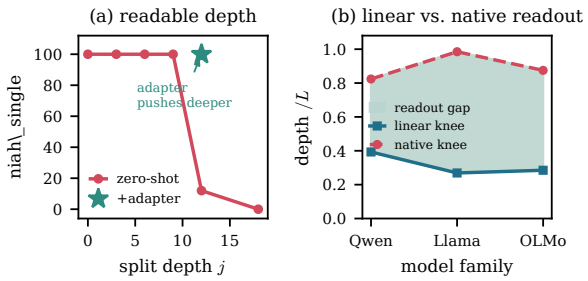

Figure 2: **Intermediate accessibility motivates, but does not validate, depth reuse.** (a) Zero-shot task readout remains usable only up to a depth-dependent boundary. (b) Frozen linear probes become informative earlier than native LM-head readout under the specified protocols. Exact definitions and values are in Appendix A.1.

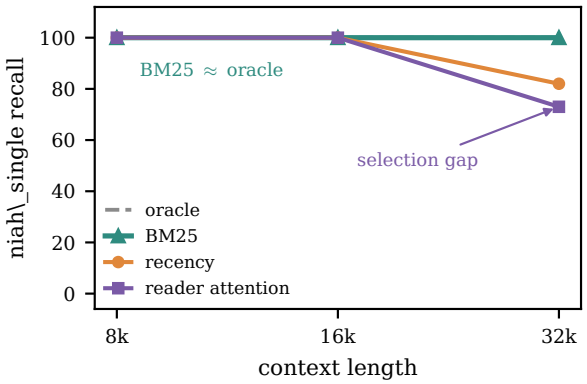

Figure 3: **Selection is one bottleneck, not always the bottleneck.** Oracle and BM25 remain strong on the single-needle diagnostic; other analyses separate evidence recall from reader accuracy and show substantial residual readout loss on multi-step tasks.

continuation under the same evidence pack.

3 Design Evidence

CoMem does not require a claim that a model "understands" a document at one particular layer. We use three empirical observations to motivate the interface: information can be accessible before native final-head readout; resuming too deep creates a sharp fidelity loss; and the dominant bottleneck can move between retrieval and residual readout depending on the task.

3.1 Accessibility precedes native readout

Under our probe protocols, semantic labels become linearly accessible before a fixed final norm and LM head can express the corresponding native answer (Figure 2). This ordering appears across the tested Qwen, Llama, and OLMo backbones. It is readout-dependent evidence, not a causal boundary or a statement that later layers are unnecessary.

3.2 The split depth is a fidelity knob

A frozen residual interface degrades as the split moves deeper. Separate self-distilled adapters shift the usable frontier but do not eliminate it: RULER changes from 98.29 at j=6 to 96.07 at j=12 and 55.41 at j=18, while fixed-pack Read speedup rises from 1.166× to 1.403× and 1.807× (Appendix Table 8). Because adapter spans and parameter counts differ, this is a deployment frontier, not a compute-matched causal depth ablation.

3.3 Retrieval and readout are separate failure modes

On a lexical single-needle diagnostic, BM25 and oracle selection retain the evidence while recency and reader-attention heuristics deteriorate with length (Figure 3). That observation is deliberately narrow. On LongEval, BM25 recall is 97–100% across 8k–128k and matched j=0 replay is nearly perfect conditional on a retrieval hit, whereas frozen j=12 fails and the distilled interface recovers only 64–76% hit-conditional accuracy. On BABILong, both retrieval misses and residual readout contribute, with the balance varying by task. Thus a bounded memory needs both a suitable selector and a compatible stored representation.

3.4 Implications

The evidence suggests three design requirements. First, expose split depth as a measured quality–cost knob. Second, retrieve a bounded relevant pack but allow full upper-layer interaction among its chunks. Third, preserve enough lower-layer document context during Write that the stored residual is compatible with later composition. Section 4 instantiates the first two; Section 5 evaluates independent and overlapping Write variants for the third.

4 CoMem: Depth-Partitioned Computation Reuse

4.1 Overview and three read pathways

CoMem partitions a decoder along depth (Figure 1): Write caches one intermediate residual tensor per chunk; Select retrieves a bounded subset; Read recomputes the upper layers with the query present. Ranges are half-open, and h_j is the input to block j. The resumed computation is related

to KV-Direct (Qasim et al., 2026), but adds shallow writes, a tunable split, and bounded external retrieval.

Three read pathways. Full context runs all L layers over every token. Text RAG bounds the prompt but still runs all layers. Our matched j=0 raw-text replay control fixes CoMem's selector, pack, and mask while storing token IDs and replaying all layers. CoMem j=12 selects the same chunks, retrieves prewritten h_{12}, and runs only $[12{:}L)$. Both retrieved paths bound the token working set; only CoMem skips repeated lower-layer document computation, at the cost of residual storage and an approximate interface.

Notation and storage. Let the decoder have L layers, width d, head width d_{head}, split j, and chunk size c. CoMem stores $h_j \in \mathbb{R}^{c\times d}$. With a common dtype, its per-token storage relative to full KV is

$$\frac{|h_j|}{|\text{KV}|} = \frac{d}{2Ln_{\text{kv}}d_{\text{head}}} = \frac{n_{\text{q}}}{2Ln_{\text{kv}}}, \qquad (1)$$

where n_{q} and n_{kv} are the query- and KV-head counts, and $d = n_{\text{q}}d_{\text{head}}$. Qwen3-8B uses d=4096, L=36, n_{q}=32, and n_{kv}=8, yielding $1/18$ (8 KB vs. 144 KB per token in bf16). The residual store is about 1 GiB at 128k tokens and 8.2 GB at one million tokens: smaller than full KV, but roughly three orders of magnitude larger than token IDs or compressed text. Depth reuse is most attractive when this prepaid representation serves repeated queries. At read time we assemble a *pack* whose length is fixed by k and c,

$$|\text{pack}| = \text{sink} + k{\cdot}c + \text{query} \approx 6.7\,\text{k}, \qquad (2)$$

for k=12, c=512, and—crucially—*independent of the context length.*

Why BM25? The flagship uses iterative token-ID BM25 as a deliberately lightweight selector: it is training-free, deterministic, CPU-only, and requires no embedding model or neural forward. It is also well matched to the entity and key overlap in our diagnostics. This choice isolates the memory interface from a separately trained retriever; CoMem itself is selector-agnostic, and we do not claim that lexical retrieval is universally optimal. Appendix A.1 compares BM25 with recency, reader-attention, oracle, and a frozen dense retriever and reports their scaling limits.

Algorithm 1 CoMem: Write and Read

Require: backbone layers $[0{:}L)$, split j, chunk size c, top-k

Write (streaming, once per context):
for each c-token chunk x in the context **do**
  $h_j \leftarrow \text{layers}_{[0:j)}(x)$ {chunk-local RoPE}
  store h_j under a unique document/chunk ID
end for
Read (per query q):
$S \leftarrow$ LEXICAL-TOP-$k(q)$ {bounded top-12 flagship retrieval}
pack $\leftarrow [\text{sink};\ \{h_j\}_{s\in S};\ h_j(q)]$ {fresh contiguous RoPE}
logits $\leftarrow \text{layers}_{[j:L)}(\text{pack})$ {full cross-pack attention}
return next-token logits

4.2 Write: caching intermediate residuals

The document is streamed in disjoint c-token chunks. Each chunk restarts rotary positions (Su et al., 2021) at zero, runs through $[0{:}j)$, and writes h_j to an external store under a unique document/chunk ID. Chunk-local positions make cached states reusable regardless of their original document offset. The Write cost is linear in context length and executes only j layers. The hidden store and BM25 index also remain linear; only the model-side online Read is bounded.

Overlap-Write. Independent chunks omit lower-layer context that may be needed to form a composable residual. We therefore evaluate a deployable variant in which chunk i is prefixed during Write by the final w tokens preceding that chunk; only the residuals for the current c tokens are persisted. The prefix is discarded, so persistent bytes/token, selected pack length, and Read/decode work are unchanged. Setting w=0 recovers the independent writer exactly. We evaluate this variant as a focused mechanism repair rather than as the flagship on every benchmark.

4.3 Depth split and self-distillation

The j knob. Split depth controls prepaid Write work versus per-query Read work. At j=0, the matched raw-text replay control performs full recomputation and matches the stock forward within $\max|\Delta\text{logit}| < 10^{-4}$ on that same pack; at j=L, no upper layer is replayed. Intermediate j trades cheaper Read for query-blind readout loss. Self-distillation enables the flagship operating point

j=12/36. KV-Direct is the separate no-retrieval full-context arm.

Self-distillation. Following knowledge distillation and low-rank adaptation (Hinton et al., 2015; Hu et al., 2022), we train a student at j=12 to match a frozen j=0 teacher on the teacher's top-64 logit support. Let p and q be their renormalized distributions on that support. The actual objective is the bidirectional KL

$$\mathcal{L}_{\text{distill}} = 0.6\,\mathrm{KL}(p\|q) + 0.4\,\mathrm{KL}(q\|p), \quad (3)$$

with no auxiliary cross-entropy term. The canonical adapter trains rank-32 LoRA (α=64, zero dropout) on all attention and MLP projections in layers 12–35 of a frozen Qwen3-8B. Training uses 4,000 steps of 8-GPU bf16 DDP (effective batch 8), AdamW with peak learning rate 8×10^{-5}, 100 warmup steps and cosine decay, on 2,048-token windows from the PG-19 training split (Rae et al., 2019). No synthetic long-context examples, task supervision, retrieval, or CE loss is used. This pushes the usable cache depth deeper and transfers without benchmark-specific supervision across the evaluated tasks (§5).

The backbone remains frozen. The j=12 flagship uses this LoRA, while a shallower adapter-free arm isolates the inference architecture. Designing compressible cache points during pretraining is complementary and outside our scope.

4.4 Read: bounded retrieval and recompute

The read pack. A selector returns top-k chunk indices. Their hiddens are ordered by document position and packed as `[sink; selected` h_j`; query` h_j`]` with fresh, contiguous RoPE. Layers $[j{:}L)$ then run with full causal attention, allowing later chunks and the query to integrate earlier evidence. For fixed k and c, model-side Read FLOPs and KV working memory are independent of stored-context length; Table 14 tests the full-attention choice.

The sink is the backbone's BOS token passed through layers $[0{:}j)$ as a one-token chunk. For generation, the query is first written with a lower-band KV cache, while the packed sink, selected states, and query are prefetched once through $[j{:}L)$ to create an upper-band cache. Each generated token then runs through $[0{:}j)$ at the next query-local position using the lower cache, and its result runs through $[j{:}L)$ at the next packed position using the upper cache; both position counters advance by one. The retrieved pack is never rerun per token. Decode work is therefore independent of stored-context length for a fixed pack and generated prefix. Appendix A.4 reports the generation timing path and component boundaries.

Bounded lexical retrieval. BM25 (Robertson and Zaragoza, 2009) returns a bounded top-k set. The flagship uses k=12, hop width four, and three automatic frontier-expansion rounds with deduplication. Oracle, recency, reader-attention, and wider retrieval are analysis-only controls reported in §5 and Appendix A.1.

5 Experiments

5.1 Experimental setup

Model and interface. The principal backbone is Qwen3-8B (Yang et al., 2025) (36 layers; exact revision in Appendix A.4). Unless stated otherwise, CoMem uses j=12, 512-token chunks, iterative BM25, top-12 retrieval, a BOS sink, and a rank-32 self-distillation LoRA on layers 12–35; the backbone is frozen. All methods use plain-text prompting with chat templating disabled. The selected pack is approximately 6.5k tokens.

Evaluation. We report RULER (Hsieh et al., 2024), BABILong (Kuratov et al., 2024), LongEval (Li et al., 2023), six LongBench QA datasets (Bai et al., 2023), and LoCoMo (Maharana et al., 2024). Distinct RULER cohorts are labeled and never pooled. LoCoMo uses the full 1,986-example set; semantic Judge is primary, with item and conversation-cluster bootstrap intervals plus an independent-judge audit. Appendices A.2–B give every cell, sample count, prompt, aggregation rule, and integrity check.

Efficiency protocol. We distinguish (i) selected-pack model Read, (ii) select-first online prefill after a persistent store exists, (iii) Write-inclusive pipeline time, and (iv) external-store/index costs. Hardware and timing boundaries are reported with each table; numbers from different cohorts are not mixed into a causal speedup.

5.2 The matched depth-reuse frontier

Table 2 is the central comparison. Raw-text replay (j=0) and CoMem receive the same selected chunks in the same order and use the same sink, mask, examples, and LoRA. Skipping the lower 12 layers reduces fixed-pack Read from 931.9 to

Operating point	Stored object	Replay	RULER↑	Read	Write	Store/token
Matched raw-text replay (j=0)	token IDs	36 layers	99.19	931.9 ms	on query	∼4–8 B
CoMem + LoRA (j=12)	h_{12}	24 layers	96.07	664.4 ms	once	8,192 B
Continuous-prefix control	h_{12}	24 layers	99.19	—	non-deployable	8,192 B

Table 2: **Matched depth-reuse trade-off on Qwen3-8B.** Quality uses the paired 15-cell RULER-B cohort (n=1,500); the j=0–j=12 gap is 3.12 points with paired-bootstrap 95% CI $[2.36, 3.93]$. Read latency uses the same LoRA and a fixed approximately 6.5k-token pack retrieved from 16k source contexts (three process medians); retrieval, persistent I/O, reusable Write, and decode are excluded. The continuous-prefix control supplies layer-12 states from a single causal lower-layer forward and exactly matches raw-text replay, showing that upper-layer continuation can be faithful with compatible states. The control jointly restores document, cross-chunk, and query-side lower-layer interaction, so it is an implementation/fidelity ceiling rather than a single-factor attribution.

664.4 ms (1.403×), but lowers the RULER macro from 99.19 to 96.07. CoMem is therefore not quality preserving. The continuous-prefix control exactly matches j=0, showing that the upper-layer continuation implementation can reproduce replay when supplied states from a single lower-layer forward. Because this control jointly restores document, cross-chunk, and query-side lower-layer interactions, it does not by itself identify which interaction explains every deployable error.

Across broader benchmarks, matched j=0 is also the quality ceiling on most tasks: it scores 97.2 versus 69.0 on LongEval, 41.59 versus 38.27 on LoCoMo, and 12.31 versus 12.15 on the six LongBench QA datasets. CoMem is stronger on BABILong qa5 but weaker on qa1/qa2. These results motivate a workload question, not a dominance claim: when does avoiding repeated lower-layer computation justify the fidelity and storage costs?

5.3 When does prepaid depth reuse amortize?

The residual store costs 8,192 bytes/token in bf16—about 1 GiB at 128k and 8.2 GB at one million tokens—versus a few bytes/token for raw token IDs. Serving measurements explicitly write a document once and answer repeated queries from GPU- or CPU-resident storage. At 32k, CoMem overtakes raw-text replay after approximately 8–11 queries across the measured generation lengths; at 128k with one generated token, crossover is 25.8 queries for a GPU-resident store and 27.6 for CPU-resident storage. The advantage appears only under reuse; for one-off queries, raw-text replay is cheaper.

Table 3 reports a different operating point: the persistent store already exists and selection bounds the online working set. At 128k, CoMem prefills its 6,657-token pack in 1.10 s and 18.7 GB, versus 71.37 s and 50.0 GB for dense full-context prefill on the same H20. The 64.9× speedup combines bounded selection with depth reuse; it must not be interpreted as the incremental benefit of depth alone. A separate same-adapter, Write-inclusive L20A cohort reaches 2.74× at 128k while keeping peak memory near 18.5 GB (Appendix A.4).

Length	Dense prefill	CoMem prefill	Speedup	Dense peak	CoMem peak
8k	1.20 s	1.05 s	1.14×	18.8 GB	18.7 GB
32k	7.33 s	1.06 s	6.92×	25.0 GB	18.7 GB
128k	71.37 s	1.10 s	**64.9**×	50.0 GB	**18.7 GB**

Table 3: Select-first online prefill on one H20 (Qwen3-8B, bf16/SDPA, median of three after one warmup). Dense runs the stock full-context path; CoMem uses its rank-32 LoRA and a fixed 6,657-token top-12 pack. This cohort times online model prefill after the persistent memory is built; it excludes one-time document Write and external-store fetch. The speedup is therefore an online bounded-read operating point, not a production end-to-end or depth-only causal effect.

5.4 What limits fidelity on multikey retrieval?

The paired multikey diagnostic exposes one missing ingredient. Independent chunk writing reaches 92.5, whereas a document-contextual lower-layer control reaches 100.0 while using the same upper-layer Read interface. A 32-token left-overlap recovers six points to 98.5 with unchanged persistent storage and Read/decode work; 128 tokens reaches 99.0. A 2×2 factorization confirms that lower-layer attention scope is the larger tested effect: switching from chunk-local to document-contextual Write adds 7.5–12.0 points depending on position mode, whereas applying document-origin positions alone to chunk-local states re-

Writer	Context	Macro↑	Write FLOPs
Independent (w=0)	current chunk	92.5	1.000×
Overlap (w=32)	32-token left prefix	98.5	1.057×
Overlap (w=64)	64-token left prefix	98.5	1.115×
Overlap (w=128)	128-token left prefix	**99.0**	1.229×
Document-context control	full prefix	100.0	$O(N)$ forward

Table 4: **Lower-layer Write context recovers the deployable gap.** Paired Qwen3-8B results on RULER `niah_multikey_1` at 8k and 16k (n=200 pooled). Every overlap width significantly exceeds the independent writer; for w=32, the gain is +6.0 points, 95% CI $[3.0, 9.5]$. Persistent storage and Read/decode are identical across deployable rows. FLOPs are theoretical lower-12 Write ratios; measured wall-clock overhead is larger because of chunk-level execution.

Method	top-k	Matched quality↑	Online latency
Raw-text replay	10	**64.78**	within ±5%
CoMem + LoRA	12	53.22	anchor

Table 5: **Equal-latency BM25 comparison is negative for CoMem.** The retrieval budget is chosen from a latency-only calibration split, then evaluated on a mixed diagnostic cohort. CoMem trails raw-text replay by 11.56 points (paired-bootstrap 95% CI $[-14.44, -8.67]$). Thus the extra evidence admitted by cheaper upper-layer replay does not compensate for residual readout loss in this cohort.

duces accuracy by 4.5 points. The factors interact, so we do not assign an additive causal decomposition. Within this diagnostic, adding context during Write is substantially more useful than merely remapping positions during Read; we do not assume the same attribution holds on every benchmark.

Retrieval versus readout. The conclusion is task dependent. On LongEval, BM25 recall is 97–100% and raw-text replay is nearly perfect conditional on a hit; frozen residual replay fails, while distillation recovers only 64–76% hit-conditional accuracy. On BABILong, dense and lexical retrieval both lose recall at longer contexts and the residual reader can fail even when evidence is retrieved. These decompositions explain why widening the evidence budget is not sufficient.

5.5 Equal-latency evidence budgets

We pre-register a direct challenge: because CoMem replays fewer layers, can it read more chunks than raw-text replay at the same online latency and thereby win on quality? Under BM25, the answer is no on the evaluated cohort. Latency-only calibration selects raw-text top-10 against CoMem top-12; raw text then leads by 11.56 points. This negative result narrows the systems claim: CoMem reduces repeated computation, but its current residual interface does not uniformly convert the saved latency into superior task quality. A frozen BGE-large-en-v1.5 experiment further shows a complementary failure mode: dense-retrieval recall drops at long lengths even when the conditional reader remains strong (Appendix A.1).

5.6 Long-context and cross-model evidence

On matched n=100 RULER scaling, unextended CoMem remains usable through 128k, scoring 98/93/99.0 on single-needle, multikey, and variable tracking; KV-Direct with YaRN scores 100/89/57.8 (Appendix A.2). These are synthetic length-extension stress tests, not evidence that CoMem is uniformly more accurate inside the backbone's native window. Additional appendices report LoCoMo scale trends, standard KV compression baselines with their full-prefill cost, and sparse-MoE ports. They support portability of the interface, but the strongest controlled claims remain the Qwen3-8B matched comparisons above.

6 Conclusion

CoMem makes transformer depth an explicit cross-query reuse axis: contexts are written to intermediate residuals, a bounded set is selected per query, and only the upper layers are resumed. The resulting benefit is a frontier, not a free compression. On a matched evidence pack, depth reuse speeds model Read by 1.403× while costing 3.12 RULER points; its one-time Write amortizes only after repeated queries. Much larger full-context speedups arise when this depth reuse is combined with bounded selection. On a paired multikey diagnostic, controlled Write variants show that missing lower-layer document context is a major source of fidelity loss: a small overlap restores most of that gap without enlarging persistent storage or per-query Read. At equal online latency, however, raw-text replay still wins on our mixed diagnostic cohort. These results establish persistent intermediate residuals as a useful systems design point and identify contextual writing as a promising interface direction for future memory systems.

Limitations

CoMem is evaluated primarily on one 8B backbone with a lexical selector and an English benchmark suite. Results on other Qwen scales and sparse MoEs support portability but are not matched replications, and several large-model cells use smaller sample counts. The principal adapter is trained once; most ablations measure evaluation uncertainty rather than training-seed variance.

The system assumes repeated queries over a relatively stable corpus. Its residual store grows linearly and is much larger than raw text or token IDs; for one-off queries, frequently edited documents, or low reuse, raw-text retrieval is usually preferable. Selector and external-store latency still grow with corpus size even though model-side Read is bounded. Reported crossover and speed measurements depend on hardware, kernels, batch size, storage tier, and the precise timing boundary.

Stored states are tied to the exact backbone, tokenizer, split depth, adapter, and lower-layer implementation; model upgrades generally require rewriting the store. Overlap-Write also weakens strict chunk independence because a local edit can invalidate the following chunk's cached state. We do not evaluate quantization of residual states, cache eviction, or multi-tenant update contention.

Quality is task dependent. Bounded top-k retrieval imposes an evidence ceiling, lexical selection transfers poorly to low-overlap or non-whitespace languages, and residual readout can fail even when the relevant evidence is present. The overlap-Write repair is tested on a focused multikey diagnostic rather than the entire benchmark suite, so it should not be interpreted as a universal quality fix. Results beyond Qwen3-8B's native context window are stress tests that mix the memory interface with the backbone's positional extrapolation behavior.

Our probes and controls characterize readout compatibility; they do not locate "understanding," factual storage, or causal necessity at a particular layer. Long-form generation, code, multilingual and multimodal memory, dynamic updates/deletions, stronger learned retrievers, and production-scale multi-tenant scheduling remain open.

7 Ethical Considerations

CoMem changes the cost and storage organization of an underlying language model; it does not add a safety mechanism. It therefore inherits risks such as hallucination, biased generation, unsafe completion, and unintended disclosure of sensitive text selected from a memory store. Lower-cost access to long histories could also scale benign applications and deliberate misuse alike. Cached residual tensors may also expose source information through inversion or membership-inference attacks; they should receive protections at least as strong as the source text. We do not empirically evaluate these attacks, so the risk is a deployment concern rather than a demonstrated vulnerability. Their large persistent volume and possible cross-tenant reuse make authorization, isolation, and deletion operational requirements rather than properties conferred by non-readable tensor storage. Deployments should encrypt and audit the persistent store, enforce access control, filter or redact sensitive sources before Write, support verifiable deletion, restrict retrieval to authorized material, and retain application-appropriate output monitoring and human review. Although bounded reads reduce accelerator memory and per-query computation, training and large-scale evaluation still consume energy; we report the measured hardware and final training budget in Appendix A.4.

We collect no new human-subject data and recruit no annotators. Our experiments use previously released model, corpus, and benchmark artifacts; LoCoMo's released conversations are generated by paired language-model agents rather than newly collected private conversations. The submitted software archive contains code only and excludes model weights, benchmark text, predictions, credentials, and API responses.

References

Shubham Agarwal, Sai Sundaresan, Subrata Mitra, Debabrata Mahapatra, Archit Gupta, Rounak Sharma, Nirmal Joshua Kapu, Tong Yu, and Shiv Saini. 2025. Cache-Craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data*.

Yushi Bai et al. 2023. LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*.

Zefan Cai et al. 2024. PyramidKV: Dynamic KV cache

compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*.

Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in LLM serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*.

Yoav Gelberg, Yam Eitan, Michael Bronstein, Yarin Gal, and Haggai Maron. 2026. Training transformers for KV cache compressibility. *arXiv preprint arXiv:2605.05971*.

Junhui He, Junna Xing, Nan Wang, Rui Xu, Shangyu Wu, Peng Zhou, Qiang Liu, Chun Jason Xue, and Qingan Li. 2025. A^{2}ATS: Retrieval-based KV cache reduction via windowed rotary position embedding and query-aware vector quantization. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 12451–12463.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Cheng-Ping Hsieh et al. 2024. RULER: What's the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Chao Jin et al. 2024. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *arXiv preprint arXiv:2404.12457*.

Yuri Kuratov et al. 2024. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *NeurIPS Datasets and Benchmarks*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.

Dacheng Li, Rulin Shao, Anze Xie, Ying Sheng, Lianmin Zheng, Joseph E. Gonzalez, Ion Stoica, Xuezhe Ma, and Hao Zhang. 2023. How long can open-source LLMs truly promise on context length? LMSYS Org Blog.

Mengjie Li, Yuan Feng, Xike Xie, and William J. Song. 2026. REAL: Retrieval-reasoning and logic-constructed attention behaviors for long-context KV cache compression. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 39035–39052.

Yuhong Li et al. 2024. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*.

Manlai Liang, Mandi Liu, Jiangzhou Ji, Huaijun Li, Haobo Yang, Yaohan He, and Jinlong Li. 2025. ILRe: Intermediate layer retrieval for context compression in causal language models. *arXiv preprint arXiv:2508.17892*.

Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. 2024a. MiniCache: KV cache compression in depth dimension for large language models. *arXiv preprint arXiv:2405.14366*.

Di Liu, Meng Chen, Baotong Lu, Huiqiang Jiang, Zhenhua Han, Qianxi Zhang, Qi Chen, Chengruidong Zhang, Bailu Ding, Kai Zhang, Chen Chen, Fan Yang, Yuqing Yang, and Lili Qiu. 2024b. RetrievalAttention: Accelerating long-context LLM inference via vector retrieval. *arXiv preprint arXiv:2409.10516*.

Adyasha Maharana et al. 2024. Evaluating very long-term conversational memory of LLM agents. *arXiv preprint arXiv:2402.17753*.

Kaleem Ullah Qasim, Jiashu Zhang, Muhammad Kafeel Shaheen, Razan Alharith, and Heying Zhang. 2026. The residual stream is all you need: On the redundancy of the KV cache in transformer inference. *arXiv preprint arXiv:2603.19664*.

Hongjin Qian, Zheng Liu, Peitian Zhang, Zhicheng Dou, and Defu Lian. 2025. Boosting long-context information seeking via query-guided activation refilling. In *Proceedings of ACL*, pages 9453–9464.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. 2019. Compressive transformers for long-range sequence modelling. *arXiv preprint arXiv:1911.05507*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Zhiyuan Shi, Qibo Qiu, Xuefeng, Zhonglin Jiang, Li Yu, Jian Jiang, Xiaofei He, and Wenxiao Wang. 2026. HeteroCache: A dynamic retrieval approach to heterogeneous KV cache compression for long-context LLM inference. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, pages 43172–43187.

Woomin Song, Sai Muralidhar Jayanthi, Srikanth Ronanki, Kanthashree Mysore Sathyendra, Jinwoo

Shin, Aram Galstyan, Shubham Katiyar, and Sravan Babu Bodapati. 2025. Compress, gather, and recompute: REFORMing long-context processing in transformers. *arXiv preprint arXiv:2506.01215*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*.

Yutao Sun et al. 2024. You only cache once: Decoder-decoder architectures for language models. *arXiv preprint arXiv:2405.05254*.

Sijun Tan, Xiuyu Li, Shishir Patil, Ziyang Wu, Tianjun Zhang, Kurt Keutzer, Joseph E. Gonzalez, and Raluca Ada Popa. 2024. LLoCO: Learning long contexts offline. *arXiv preprint arXiv:2404.07979*.

Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning*.

Tencent Hunyuan Team. 2026. Hy3: An open 295b mixture-of-experts model with 21b activated parameters. https://huggingface.co/tencent/Hy3.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008.

Dongwei Wang, Zijie Liu, Song Wang, Yuxin Ren, Jianing Deng, Jingtong Hu, Tianlong Chen, and Huanrui Yang. 2025. FIER: Fine-grained and efficient KV cache retrieval for long-context LLM inference. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 9702–9713.

Qingyue Wang et al. 2023a. Recursively summarizing enables long-term dialogue memory in large language models. *arXiv preprint arXiv:2308.15022*.

Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023b. Augmenting language models with long-term memory. In *Advances in Neural Information Processing Systems*.

Yu Wang et al. 2024. MemoryLLM: Towards self-updatable large language models. *arXiv preprint arXiv:2402.04624*.

Haoyi Wu and Kewei Tu. 2024. Layer-condensed KV cache for efficient inference of large language models. *arXiv preprint arXiv:2405.10637*.

Guangxuan Xiao et al. 2024. Efficient streaming language models with attention sinks. In *ICLR*.

Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina, Mohammad Shoeybi, and Bryan Catanzaro. 2024. Retrieval meets long context large language models. In *International Conference on Learning Representations*.

An Yang et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Xintong Yang, Hao Gu, Binxing Xu, Lujun Li, Bei Liu, Jiacheng Liu, Qiyuan Zhu, Yike Guo, and Sirui Han. 2026. IndexMem: Learned KV-cache eviction with latent memory for long-context LLM inference. In *International Conference on Machine Learning*.

Jiayi Yao et al. 2024. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. *arXiv preprint arXiv:2405.16444*.

Howard Yen, Tianyu Gao, and Danqi Chen. 2024. Long-context language modeling with parallel context encoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Zhiyuan Zeng, Qipeng Guo, Xiaoran Liu, Zhangyue Yin, Wentao Shu, Mianqiu Huang, Bo Wang, Yunhua Zhou, Linlin Li, Qun Liu, and Xipeng Qiu. 2024. Memorize step by step: Efficient long-context prefilling with incremental memory and decremental chunk. In *Proceedings of EMNLP*, pages 21021–21034.

Peitian Zhang et al. 2024. Long context compression with activation beacon. *arXiv preprint arXiv:2401.03462*.

Zhenyu Zhang et al. 2023. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *NeurIPS*.

Meizhi Zhong, Xikai Liu, Chen Zhang, Yikun Lei, Yan Gao, Yao Hu, Kehai Chen, and Min Zhang. 2025. ZigZagKV: Dynamic KV cache compression for long-context modeling based on layer uncertainty. In *Proceedings of COLING*, pages 8897–8907.

A Additional Results and Ablations

This appendix contains the detailed task tables, mechanism ablations, and cross-scale generality results summarized in the eight-page main paper. Appendix B separately documents aggregation, uncertainty, and the LoCoMo judge protocol.

Content-depth probe protocol. For SST2, WiC, and RTE we extract every layer once from a fixed stratified pool (up to 3,000 train examples), then fit L2 logistic regressions on five independent 60/20/20 train/dev/test splits. Features are standardized; $C \in \{0.1, 1, 10\}$ is selected on development data. For layer accuracy $a(l)$, the knee is $\min\{l : a(l) \geq 0.98 \max_l a(l)\}$. The native readout instead applies the model's own final norm and LM head to each layer using fixed verbalizers. Lexical uni/bi-grams, token length, random labels, majority class, and balanced training serve as controls. On SST2, probe peak is about 0.90 versus 0.71 lexical, 0.56 position/majority, and 0.54 random-label accuracy; WiC/RTE are lower-selectivity and have wider per-task intervals. The reported confidence intervals are Student-t intervals over task–split knees, not training-seed uncertainty.

A.1 Mechanism ablations

Split depth and self-distillation. Table 6 reports linear and native-readout knees over five stratified data splits for Qwen and OLMo, plus a matched-support Llama SST2 result; protocol details follow below. Table 7 gives the frozen $j = 0/6/9/12$ quality sweep and same-j distilled control; Table 8 gives separately distilled $j = 6/9/12/18$ deployment points; Table 9 combines quality, storage, and same-H20 cost for the four core operating points. Table 10 provides a separate same-j RULER control. Finally, Table 11 uses an HCache-style checkpoint path only as an interface-adaptation diagnostic, showing that the readout repair transfers beyond CoMem.

Selection. Table 12 compares oracle, BM25, recency, and reader attention. BM25 is near oracle on the lexical needle tasks and is more reliable than the tested training-free alternatives at long lengths. Variable tracking instead requires iterative expansion along literal variable links; a three-round CPU microbenchmark costs 188.8 ms at 128k, versus 41.0 ms for one-shot BM25, without an additional model forward. This is a low-overhead selector in the tested regime, not a constant-time one: lookup grows approximately linearly with store size (Table 23). Dense or learned retrieval remains untested, so our results establish a transparent lexical operating point rather than selector optimality.

Read interaction and chunk size. Table 14 compares full cross-chunk attention with block-diagonal reuse. Table 15 reports the chunk-size sweep; its entries are point estimates rather than a multi-seed stability ranking.

Model	L	linear knee ($/L$, 95% CI)	native knee ($/L$)	gap (Δ/L)
Qwen3-8B	36	0.393 [0.319,0.466]	0.824	0.43
Llama-3-8B†	32	0.275	1.000	0.725
OLMo-2-7B	32	0.285 [0.214,0.357]	0.875	0.59

Table 6: **Linear accessibility precedes native readout under these probes.** The linear knee is the first layer reaching 98% of peak held-out probe accuracy; the native knee applies each model's own final norm and LM head at every layer. Qwen and OLMo aggregate SST2, WiC, and RTE over five stratified data splits; their intervals reflect task–split variation, not model-training seeds. †For Llama, WiC and RTE native verbalizers remain near chance, so we report the SST2-only matched-support knees and do not form a misleading three-task native aggregate. These correlational probes do not localize understanding or establish the same knees on long-memory tasks.

Configuration	Split j	LoRA	LoCoMo Judge	BABILong mean	
				qa1 / qa2	qa5
Matched raw-text replay	0	no	**41.59**	66.3 / 38.9	63.3
CoMem frozen	6	no	32.78	44.6 / 22.6	53.9
CoMem frozen	9	no	29.15	42.4 / 19.6	55.6
CoMem frozen	12	no	24.52	33.4 / 18.0	60.3
CoMem distilled	12	yes	38.27	55.6 / 27.0	**68.7**

Table 7: Controlled comparison of retrieval, split depth, and self-distillation using Qwen3-8B. All configurations follow the same plain-text prompting protocol, with model-specific chat templating disabled (`chat_template=False`), and retrieve the same top-12 pack. For the frozen configurations, readout fidelity decreases as the split moves deeper. At the same split j=12, the rank-32 LoRA improves the LoCoMo Judge score by 13.75 points and the BABILong qa1, qa2, and qa5 means by 22.2, 9.0, and 8.4 points, respectively. BABILong entries are rounded task means from the underlying 21 task–length cells; the raw-cell j=0 macro is 56.14. The j=0 configuration performs full-depth recomputation over the retrieved pack, providing a selected-pack reference without lower-layer reuse and thus no depth-based reduction in query-time computation.

Split j	LoRA params	RULER B	Full − deployed (95% CI)	LoCoMo	Read (ms)	Read speedup	Write (ms)
6	72.745M	98.29	$0.91\ [0.49, 1.39]$	40.38	830.3	$1.166\times$	133.4
9	65.470M	97.55	$1.65\ [1.04, 2.31]$	39.02	748.3	$1.273\times$	196.9
12	58.196M	96.07	$3.12\ [2.36, 3.93]$	38.27	664.4	$1.403\times$	—
18	43.647M	55.41	$43.79\ [41.77, 45.85]$	28.65	499.5	$1.807\times$	390.3

Table 8: Distilled depth curve. Each split has a separately trained rank-32 adapter on layers $[j{:}36)$ under the same data order, seed, steps, optimizer, and KL recipe. RULER uses Cohort B (n=1,500 paired examples); gap CIs are paired bootstrap intervals against full replay with that depth's adapter, and all McNemar tests reject equality ($p \leq 3.05 \times 10^{-5}$). LoCoMo reports the full-set GPT-4o judge point estimate (n=1,986). Read/Write use a fixed 16k same-pack input and three process medians; retrieval, I/O, and decode are excluded, and total latency remains decode-dominated. The matched fixed-pack Write value was not retained for the earlier j=12 artifact. Adapter spans and parameter counts differ, so this is not a training-compute-matched depth ablation.

Operating point	RULER	B/token	Query (s)	Write (s)	Peak (GB)
Full context	78.80^{A}	147,456	50.59	0	$50.0^{\dagger}$
Matched raw-text replay (j=0)	$\mathbf{99.20}^{B}$	∼4–8	1.141	0.09	18.5
CoMem frozen (j=12)	8.01^{A}	8,192	0.849	5.83	18.5
CoMem + LoRA (j=12)	96.07^{B}	8,192	**0.807**	5.83	18.5

Table 9: Representative quality and systems operating points, not a single matched Pareto frontier. Quality is a 15-cell RULER macro from the explicitly marked A/B cohort; systems columns are measured at 128k on one H20 (bf16/SDPA, median of three). Superscripts follow Table 21; the frozen quality point is not compared numerically across cohorts, while its direct same-task collapse appears in Table 10. All three retrieved systems rows use the same top-12 working set. The j=0 row stores raw token IDs and replays all layers; the j=12 rows store residuals and replay only the upper layers. The j=0 row has no LoRA whereas the flagship j=12 row is adapted; their 0.33 s/query difference is therefore a deployable operating-point comparison, not depth-only causal isolation. It prepays 5.74 s of Write and gives a 17–20-query analytic break-even in this component cohort; the separate write-once serving experiment reports length- and storage-dependent measured crossover. Frozen j=12 shows that depth reuse requires interface adaptation. †Full-context KV-cached generation peaks at 50.0 GB; a separate single-forward write-all harness that materializes full-sequence logits OOMs on H20. The 89.39 GB L20A cohort in Appendix A.4 is therefore not a contradictory hardware capacity claim.

Task	Adapter	8k	16k	32k
niah_single_2	off	36	7	22
	on	**100**	**100**	**100**
niah_multikey_1	off	8	3	1
	on	**91**	**95**	**92**

Table 10: Separate controlled RULER self-distillation ablation on Qwen3-8B at the same split j=12 (n=100 per cell). Each on/off pair uses the same task, length, selector, and cohort, so the difference isolates the adapter. Table 7 provides the broader same-j LoCoMo/BABILong control used in the main argument.

Configuration	LoRA	LoCoMo Judge
HCache-style (j=12)	no	13.29
HCache-style (j=12)	yes	**31.17**

Table 11: Interface-adaptation diagnostic on an HCache-style, retrieval-free checkpoint path. The two configurations share the j=12 split and evaluation harness, isolating the adapter. The gain shows that aligning intermediate states to native upper-layer readout transfers beyond CoMem; this is not a head-to-head long-term-memory comparison.

Task	Len	BM25	Recency	ReaderAttn	Oracle
niah_single	8k	100	100	100	100
	16k	100	100	100	100
	32k	100	82	73	100
niah_multikey	8k	99	98	97	100
	16k	99	88	90	100
	32k	99	54	60	100
var-track	8k	99.4	99.2	99.8	N/A
	16k	92.6	92.4	92.4	N/A
	32k	32.0	41.2	27.8	N/A

Table 12: Single-pass selector sweep on RULER (n=100) without a chat template; each entry is the peak over $k \in \{4, 8, 12, 16, 24\}$. Oracle retrieval keeps both needle tasks at 100, showing that their long-range gap is selection rather than readout loss. The variable-tracking rows belong to this specific selector-sweep configuration and should not be conflated with the focused top-12 flagship in Table 13. Oracle is undefined for a multi-chunk reference chain and is therefore not reported there.

Method	RULER task	8k	16k	32k	64k	128k
CoMem	niah_single_2	100.0	100.0	99.0	99.0	100.0
	var-track	96.6	97.6	98.8	99.0	95.8
StreamingLLM	niah_single	85.0	36.0	20.0	10.0	2.0
	var-track	41.8	2.2	0.6	0.2	0.8

Table 13: Equal-budget, no-chat RULER comparison (n=100/cell). CoMem and StreamingLLM use the same nominal budget and scorer but not paired examples; CoMem retrieves about 6.5k relevant tokens, whereas StreamingLLM keeps recent tokens.

Task	Full attn.	Block-diag.	Δ
RULER niah_single	100 / 100	100 / 96	0 / +4
RULER niah_multikey	**96 / 94**	60 / 32	+36 / +62
BABILong qa2	36 / 20	17 / 13	+19 / +7
BABILong qa5	78 / 69	78 / 55	0 / +14

Table 14: Full cross-chunk attention versus block-diagonal reuse (8k/16k, no chat). RULER uses n=50 and BABILong n=100 with identical top-12 retrieval. Joint recomputation matters most for multi-fact tasks.

chunk	read tokens	uncached step (s/token)	8k	16k	32k	64k
128	1,665	0.19	91.0	90.0	81.0	85.0
256	3,329	0.37	80.0	90.0	90.0	**94.0**
512	6,657	0.72	89.0	**95.0**	**97.0**	**94.0**
1024	13,313	1.60	**100.0**	89.0	92.0	**94.0**

Table 15: Chunk-size trade-off on RULER `niah_multikey` (n=100, no chat). Packed-step cost grows approximately linearly with read length; accuracy values are point estimates, not a multi-seed claim.

A.2 Full chat-template-free benchmark comparisons

Tables 16–19 give the full audited comparisons for RULER, LongEval, BABILong, and LongBench. Main rows use the unified no-chat protocol; MemoryLLM is the external persistent-memory reference using its released Llama-3 checkpoint, while the marked LLoCO row uses its released native setup. The all-method LoCoMo comparison is Table 22; judge details are in Appendix B.

Method	Task	8k	16k	32k	64k	128k
CoMem + LoRA	single-2	100	100	99	99	100
	multikey	95	94	97	91	93
	var-track	96.6	97.6	98.8	99.0	95.8
CoMem frozen (j=9)	single	96	99	99	99	96
	multikey	44	44	59	30	48
	var-track	45	38.8	36.4	31.2	25.8
KV-Direct	single	100	100	100	100	0
	multikey	100	100	98	88	0
	var-track	100	100	99.8	96.2	0
InfLLM	single	100	100	92	61	57
	multikey	100	79	65	45	20
	var-track	100	96.8	90.6	81.8	79.2
StreamingLLM	single	85	36	20	10	2
	multikey	83	34	18	13	4
	var-track	41.8	2.2	0.6	0.2	0.8
External persistent-memory reference (different backbone)						
MemoryLLM[†]	single	29	40	37	21	21
	multikey	28	24	25	12	10
	var-track	0	1.2	0	0	0

Table 16: Full chat-template-free RULER comparison using official `string_match_all` (n=100 per cell). KV-Direct places the full context in the read with native RoPE and no YaRN; its 64k/128k cells are unextended stress references rather than a length-extended upper bound. CoMem keeps a bounded retrieved pack. This is **Cohort A**: every method uses `niah_single_2`, `niah_multikey_1`, and `variable_tracking`, with n=100 and the official scorer. Its CoMem macro is 97.05. Cohort B instead uses `niah_single_3` to match the YaRN arm (Table 20) and yields 96.07; the cohorts are never pooled in a direct comparison. [†]MemoryLLM is the principal external persistent-memory reference and uses its released Llama-3-8B-chat checkpoint; its different backbone precludes matched attribution.

Method	8k	16k	32k	64k	128k	Mean
CoMem + LoRA	69	75	64	**67**	**70**	**69.0**
CoMem frozen (j=9)	8	0	4	0	4	3.2
KV-Direct	**100**	**96**	**92**	38	0	65.2
StreamingLLM	86	34	18	10	6	30.8
InfLLM	60	30	12	4	2	21.6
MemoryLLM†	22	22	16	6	2	13.6

Table 17: LongEval register-content accuracy. Unmarked rows use the unified chat-template-free protocol. Means use the common 8k–128k support. KV-Direct uses native RoPE without YaRN, so its 64k/128k cells are unextended stress references and the five-length mean is not a length-extended upper bound. CoMem's separate six-length headline is 72.83 after including its 4k score of 92. The frozen CoMem control used 48 generation tokens rather than 16; both budgets exceed the single-number answer length. †MemoryLLM is the external persistent-memory reference and uses a different chat-tuned backbone, so its row is not a matched comparison.

Method	Task	0k	1k	2k	4k	8k	16k	32k	Mean
CoMem + LoRA	qa1	**98**	80	68	68	46	17	12	55.6
	qa2	26	44	43	44	23	8	1	27.0
	qa5	68	**76**	**76**	**75**	68	**60**	**58**	**68.7**
CoMem frozen (j=9)	qa1	**98**	59	71	57	7	1	4	42.4
	qa2	53	17	28	26	8	4	1	19.6
	qa5	70	74	65	60	41	39	40	55.6
KV-Direct	qa1	**98**	**84**	80	**74**	**80**	**72**	**63**	**78.7**
	qa2	**58**	53	**50**	46	**49**	**49**	**37**	**48.9**
	qa5	71	73	62	59	65	42	**58**	61.4
InfLLM	qa1	97	**84**	**81**	72	69	52	34	69.9
	qa2	51	**58**	41	47	41	39	30	43.9
	qa5	69	68	63	66	**73**	**60**	55	64.9
StreamingLLM	qa1	97	83	80	71	23	27	12	56.1
	qa2	49	56	44	**49**	19	11	4	33.1
	qa5	68	65	68	65	39	47	34	55.1
MemoryLLM†	qa1	52	46	35	28	23	17	12	30.4
	qa2	37	29	17	18	21	17	11	21.4
	qa5	50	45	39	35	35	34	29	38.1

Table 18: BABILong accuracy (n=100 per task–length cell), evaluated with the official `compare_answers` scorer and `TASK_LABELS`. Within the native context window, full-context KV-Direct is substantially stronger on qa1 and qa2, exposing the compression tax, whereas distilled CoMem achieves the highest qa5 mean, ahead of InfLLM (68.7 vs. 64.9). All methods are evaluated under the unified plain-text prompting protocol, with model-specific chat templating disabled (`chat_template=False`). †MemoryLLM is the principal external persistent-memory reference and uses its released Llama-3-8B-chat checkpoint. Its different backbone precludes matched attribution; the official chat template does not improve these diagnostics (Appendix A).

Method	NQA	Qasper	Hotpot	2Wiki	MultiF.	Musique	Macro
CoMem + LoRA	4.12	11.01	11.62	**12.83**	**25.41**	**7.91**	12.15
CoMem frozen (j=9)	**4.63**	11.23	9.41	10.71	22.05	5.72	10.63
KV-Direct	3.70	**11.82**	**12.68**	12.03	25.30	7.49	**12.17**
InfLLM	2.99	11.35	12.08	12.50	25.33	6.94	11.86
StreamingLLM	3.49	11.64	11.02	12.19	22.42	5.88	11.11
MemoryLLM†	3.13	8.46	8.76	10.27	17.43	5.98	9.01
LLoCO‡	24.21	24.45	44.24	—	—	—	30.97

Table 19: Official LongBench QA token F1 by dataset. Except for the explicitly marked LLoCO diagnostic, methods use the same plain-text no-chat protocol. CoMem with LoRA scores 12.15 and full-context KV-Direct scores 12.17; because only CoMem uses the split-interface adapter, this near-equality is descriptive rather than a training-budget-matched compression estimate. The frozen CoMem configuration reports adapter-free performance at j=9. NQA denotes NarrativeQA, and MultiF. denotes MultiFieldQA-en. †MemoryLLM is the principal external persistent-memory reference and uses its released Llama-3-8B-chat checkpoint. Its different backbone precludes matched attribution; its official chat template does not improve the tested diagnostics (Appendix A). ‡LLoCO uses LLaMA-2-7B AutoCompressor, released per-domain supervised LoRAs, fp16, and its native task prompts. Released weights cover only NarrativeQA/Qasper/HotpotQA on our 200-example sets; 30.97 is their three-task macro and is not comparable to the six-task macro column.

Task	Method	8k	16k	32k	64k	128k
single needle	KV-Direct	100	100	100	100	0
	KV-Direct+YaRN†	100	100	100	100	**100**
	CoMem+LoRA	100	91	97	98	98
	CoMem+LoRA+YaRN†	100	90	97	97	98
multikey	KV-Direct	100	100	98	88	0
	KV-Direct+YaRN†	98	94	96	91	89
	CoMem+LoRA	94	91	99	90	**93**
	CoMem+LoRA+YaRN†	82	85	94	88	**93**
variable tracking	KV-Direct	100	100	99.8	96.2	0
	KV-Direct+YaRN†	99.2	99.4	26.6	67.2	57.8
	CoMem+LoRA	96.2	98.0	98.2	98.6	**99.0**
	CoMem+LoRA+YaRN†	88.6	90.0	89.4	95.0	93.6

Table 20: Matched-n=100 Qwen3-8B RULER scaling. †YaRN factor 4 gives an effective 163,840-token window. At 128k, unextended-backbone CoMem+LoRA scores 98/93/99.0 on single-needle, multikey, and variable tracking, versus 100/89/57.8 for KV-Direct+YaRN. Applying YaRN to CoMem changes these to 98/93/93.6, showing an in-window position-scaling tax on variable tracking.

Method	RULER↑ 15-cell	LongEval↑ 8k–128k	LongBench↑ 6-QA F1	BABILong↑ qa1/2/5 macro	LoCoMo Judge↑ full set
CoMem + LoRA (j=12)	**97.05**^{A} / 96.07^{B}	**69.0**	12.15	50.43	**38.27**
KV-Direct	78.80^{A}	65.2	**12.17**	**63.00**	34.59
InfLLM	77.83^{A}	21.6	11.86	59.52	22.21
StreamingLLM	23.37^{A}	30.8	11.11	48.14	25.63
MemoryLLM†	16.55^{A}	13.6	9.01	30.00	16.11
Selected-pack full-depth replay reference (excluded from ranking)					
Matched raw-text replay (j=0)	99.20^{B}	97.2	12.31	56.14	41.59

Table 21: Cross-benchmark results under the unified no-chat protocol. Bold marks the best same-backbone point estimate; RULER ranking uses cohort A only. †MemoryLLM uses a released 7B backbone with a fixed latent pool; it is listed as an external persistent-memory reference and is not budget-matched. The j=0 row retrieves the same raw-text chunks and recomputes all layers, providing the matched full-depth endpoint of CoMem's depth axis. RULER superscripts denote distinct n=100 cohorts, which are never pooled. Table 10 isolates interface adaptation; Table 20 gives the length-extension comparison.

Method	Judge	Judge$_{1:4}$	F1	Substr. acc.
CoMem + LoRA (j=12)	**38.27**	48.64	**9.15**	**23.36**
KV-Direct	34.59	43.83	9.02	22.36
StreamingLLM	25.63	31.04	7.67	13.75
InfLLM	22.21	26.43	7.39	13.34
MemoryLLM†	16.11	20.65	5.91	9.52

Table 22: LoCoMo comparison on all 1,986 chat-template-free examples. Judge$_{1:4}$ is GPT-4o accuracy on the 1,540 answerable items; the full Judge column folds in local abstention scoring for 446 adversarial items. F1 and substring accuracy are deterministic lexical diagnostics; semantic Judge is the primary metric. All rows use the same scoring protocol. The same-j adapter effect is isolated in Table 7. On paired answerable items, distilled CoMem exceeds KV-Direct by 4.81 points (item-bootstrap 95% CI $[2.34, 7.34]$); a 10-conversation cluster bootstrap gives $[1.27, 8.32]$ and 8/10 observed conversation differences favor CoMem. An independent DeepSeek-V3 audit preserves the ordering. †MemoryLLM is the external persistent-memory reference and uses its official chat-tuned Llama-3 checkpoint; its different backbone precludes matched attribution. Its native chat template does not improve this result.

A.3 Store scaling and failure boundaries

Table 23 tests the "bounded read over extensible store" claim directly. Retrieval is invariant when the required evidence fits top-12, but lookup and index construction still scale with store size. Beyond the budget, recall is physically capped; co-keyed multivalue evidence falls below even this cap (recall 0.62 at $E{=}8$), and a common-word-frequency task scores zero as coverage falls from 4.5% at 128k to 1.2% at 512k. These negative results delimit the claim: bounded retrieval is not global aggregation, and successful retrieval does not guarantee multi-hop reasoning.

We audited the PG-19 adapter corpus against the InfiniteBench long-book support using normalized 13-gram containment sketches. The conservative 0.8 threshold flags 24/86 books (163/580 records), but containment is sharply bimodal: 31 books are below 0.10, no book lies in 0.18–0.60, and 54 are at least 0.60. Any cut in 0.20–0.60 therefore flags 54/86 books and 387/580 records; anonymized character names make the 0.8 count an undercount. Existing predictions were unavailable for a strict-clean-subset rescore (113 QA and 76 choice records), so we remove the book-quality comparison rather than treat it as out-of-domain evidence. This does not affect the synthetic store-scaling results above. We did not complete equivalent overlap audits for every natural benchmark, including NarrativeQA; those results should therefore be interpreted as benchmark performance rather than contamination-free generalization estimates.

A.4 Reproducibility details

Tables 24–26 record the configuration, training, and evaluation settings used by the Qwen3-8B flagship. The archived configuration files and evaluation scripts accompany the code release.

Artifacts, licenses, and data scope. The study evaluates English text only. It uses the frozen released Qwen/Qwen3-8B checkpoint (Apache-2.0); we do no backbone continued pretraining. It also uses PG-19 (Rae et al., 2019), RULER (Hsieh et al., 2024), BABILong (Kuratov et al., 2024), LongBench (Bai et al., 2023), LongEval/LongChat (Li et al., 2023), and LoCoMo (Maharana et al., 2024). RULER and LongChat code are Apache-2.0; the LongBench repository is MIT-licensed, while its component datasets retain their upstream terms; BABILong code is Apache-2.0 and incorporates BSD-licensed bAbI tasks; LoCoMo is CC BY-NC 4.0. PG-19 contains pre-1919 Project Gutenberg books, for which the dataset page does not assert one uniform corpus license, so we do not redistribute its text. We likewise redistribute no model weights, benchmark examples, predictions, or API responses. The anonymous code archive contains source, documentation, pinned environment requirements, and third-party notices; users obtain all external artifacts from their original providers and remain responsible for their terms. Table 26 reports the evaluated supports and sample counts; LoCoMo category denominators are in Appendix B.

Software environment. The archived environment specifies Python 3.10+, PyTorch 2.10, Transformers 5.5.4, PEFT 0.10+, Datasets 2.14+, bf16, and SDPA. Evaluation uses the benchmark-specific official scorers named in Table 26; the archive records all non-default flags and provides a CPU correctness gate.

Compute accounting. The final adapter run used one node with 8 H20 GPUs for about 22 minutes, or approximately 2.9 H20 GPU-hours. Table 25 and this section report model size, trainable parameters, hardware, and measured timing protocol. Total GPU-hours across preliminary probes, failed runs, baseline generation, and all ablations were not consistently logged; we report this omission rather than extrapolating an unreliable total.

Efficiency measurement. The Write-inclusive cohort uses one NVIDIA L20A with PyTorch 2.10, Transformers 5.5.4, bf16, SDPA, and the rank-32 LoRA enabled, reporting medians of three repetitions. Full-context prefill times the complete forward; CoMem times document Write, CPU retrieval, query Write, and upper-layer Read through logits. The timer excludes one-time BM25 index construction and external-store I/O; Table 30 measures the latter separately. At 128k, full-context prefill is 6.035 s and peaks at 89.39 GB, while CoMem takes 2.202 s and peaks at 18.54 GB. Cached decode uses 20 greedy steps.

The representative operating-point cohort (Table 9) is a separate one-H20 bf16/SDPA sweep; its A/B quality values are not a single matched Pareto frontier. Retrieved rows share top-12 IDs, a 6,657-token nominal pack (measured ∼6.5k), and chunk size 512. Write-all timing processes the complete

Store	NIAH recall	NIAH score	VT recall	VT score	Read tokens	BM25 (ms)
128k	1.00	100	1.00	18	6,211 / 6,463	80 / 20
256k	1.00	100	1.00	34	6,209 / 6,463	164 / 39
512k	1.00	100	1.00	40	6,209 / 6,464	328 / 81
1M	1.00	100	1.00	54	6,209 / 6,463	697 / 170
2M	1.00	90	1.00	38	6,211 / 6,463	1,407 / 357
4M	1.00	100	1.00	34	6,208 / 6,463	2,852 / 725

Table 23: Store scaling with a fixed top-12 read. Values separated by "/" are single-evidence NIAH / five-link variable tracking (VT). Evidence recall (n=20) stays perfect and the model read stays about 6.2–6.5k tokens from 128k to 4M stored tokens; BM25 lookup grows approximately linearly. Generation scores use n=10 and reveal that VT reasoning remains noisy even when all five evidence chunks are retrieved. At 128k, increasing required VT evidence from 12 to 16/24/32 gives recall 1.0 to 0.75/0.50/0.375, exactly $\min(1, 12/E)$.

Layer	Setting	Value
Backbone	Architecture	Frozen Qwen/Qwen3-8B; revision `b968826d9c46` (full hash in artifact manifest); L=36, d=4096, 32 query heads, 8 KV heads, bf16, SDPA.
Backbone	Positions	Native window 40,960; RoPE θ=10^6; no active RoPE scaling or YaRN.
Write	Cached state	Split j=12; disjoint 512-token chunks; chunk-local positions restart at zero; one bf16 residual tensor h_j per token.
Read	Pack	$[\mathrm{BOS};\ \text{selected } h_j;\ h_j(q)]$ in document order, with fresh contiguous RoPE and full causal cross-chunk attention. The nominal cap is $1 + 12{\times}512 + 512 = 6{,}657$ tokens; actual RULER means are about 6.2–6.5k because tail chunks and queries can be shorter.
Retrieval	BM25	Token-ID documents and bare-question token IDs; k_1=1.5, b=0.75, Robertson IDF with +1 smoothing; no lowercasing, stemming, or stop-word removal.
Retrieval	Iteration	`rounds=0` means automatic $\lceil k/h \rceil$ rounds, not one-shot. Each round uses newly selected chunks as the next frontier and deduplicates the selected set.
Retrieval	Width	All five benchmarks fix k=12, hop width h=4, and therefore three automatic rounds. Chunk size 512 and the BOS sink are also fixed.
Read	Mask	The pack is one causal sequence: the final query sees all chunks; later chunks may attend to earlier chunks. In the block-diagonal ablation, chunks see only themselves and the sink, while the final query still sees all chunks.
Generation	Decoding	Greedy decoding (no sampling, temperature, top-p, or beams); no chat template and no thinking mode.
Adapter-free arm	Deployment	Frozen backbone, no LoRA, j=9; all other memory and evaluation settings are held fixed.

Table 24: Flagship CoMem inference configuration. The retrieval and read configuration is frozen across all five benchmarks; only benchmark harness parameters differ.

Setting	Value
Adapter	Rank 32, α=64, dropout 0; Q/K/V/O and gate/up/down projections in layers 12–35 only; 58.20M trainable parameters (0.71%). Final weight SHA-256: `dd09cd17457c63578c0f38dab79b287ab5da6e3f14c119aedafec1c34400536f`.
Teacher and objective	The same Qwen3-8B with adapters disabled and j=0 serves as the frozen teacher. On the teacher top-64 support, $\mathcal{L} = 0.6\,\mathrm{KL}(p\|q) + 0.4\,\mathrm{KL}(q\|p)$ with implicit temperature 1 and no CE term. The loss is applied only to the query segment.
Data	Plain PG19 train text; no synthetic long-context examples, retrieval, task labels, or benchmark supervision. Training uses 2,048-token windows, formed as four 512-token chunks.
Optimization	AdamW, $\beta = (0.9, 0.95)$, weight decay 0, peak LR 8×10^{-5}, 100-step linear warmup, cosine decay to zero, gradient clipping 1.0, seed 42, bf16, no gradient checkpointing.
Schedule	4,000 steps on 8 GPUs, one sample per GPU and no gradient accumulation (global batch 8), totaling about 65.5M tokens.
Compute	One node with 8×NVIDIA H20 GPUs. Logged throughput is about 24.5 samples/s, implying roughly 22 minutes wall-clock; training peak memory was not recorded.

Table 25: Self-distillation and LoRA training configuration. The adapter name's "4k" denotes 4,000 optimization steps; the training sequence length is 2,048 tokens.

store once; Read is per query. The j=0 text-retrieval path has no adapter, whereas j=12 uses the flagship LoRA; hence the 0.33 s Read difference is an operating-point comparison rather than a depth-only effect. Its 5.74 s extra Write implies a 17–20-query analytic break-even in that earlier LoRA-on component cohort. The main serving experiment instead measures write-once/fetch/reuse directly and obtains about 8–11 queries at 32k and 26–28 at 128k; the difference comes from source length, generation length, storage tier, and timing boundary. CoMem stores 8 KB/token (about 1 GiB at 128k), far more than text but 18× less than full KV. Main timing cohorts keep the store in HBM; Table 30 separately measures placement beyond HBM, so its I/O microbenchmark is not added to model latency.

Paired quality–latency replay control. Table 28 uses a separate H20 harness. The two arms share the Qwen3-8B checkpoint, flagship LoRA SHA-256 and all 168 mounted modules, exam-

Benchmark	Tasks and support	Generation	Scoring and aggregation
RULER	Single needle, multikey, and variable tracking; 8k–128k; 100 examples/cell; 8 shards; seed 42.	48 tokens; 60 for variable tracking.	Official case-insensitive `string_match_all`; equal-weight macro over 15 cells.
LongEval	Lines retrieval; 4k–128k; 100 examples/length; 8 shards; seed 1234.	16 tokens for the flagship; 48 for frozen controls.	First numeric answer exact match; six-length macro (five-length 8k–128k macro for matched baseline tables).
LongBench	Six QA datasets; 1,150 examples total; 8 shards; seed 42.	32/64/128 tokens by dataset.	Official multi-reference token F1; equal-weight macro over six datasets.
BABILong	qa1/qa2/qa5; 0k–32k; 100 examples/cell; 4 shards; seed 42.	20 tokens.	Official `compare_answers` with `TASK_LABELS`.
LoCoMo	All 1,986 QA pairs from 10 conversations; 4 shards.	48 tokens for the main 8B comparison; 512 for the cross-scale audit.	GPT-4o judge on categories 1–4; official local abstention rule for category 5; full-set judge is the headline.

Table 26: Benchmark-level evaluation settings. Every main comparison uses `chat_template=False`. MemoryLLM is the external persistent-memory reference, evaluated with its released chat-tuned backbone.

(a) Quality at a 6,657-token/KV budget

Method	Native A macro	Full-prompt 64k/128k single	multikey	tracking	LoCoMo F1/acc
CoMem+LoRA	**97.05**	99/100	91/93	99.0/95.8	9.15/23.36
SnapKV	72.33†	100/100	94/91	80.0/84.8	9.21/22.05
PyramidKV	72.32†	100/100	93/89	88.6/86.8	9.13/22.10

(b) Full-prefill systems cost on one H20

Method	Len.	ms	GB	ms/tok
SnapKV	8k	1,213	18.6	24.7
	32k	6,342	25.7	24.6
	128k	51,520	54.3	24.7
PyramidKV	8k	1,203	18.6	24.7
	32k	6,301	25.7	26.7
	128k	51,520	54.3	26.8

Table 27: Standard selected/compressed-KV baselines at CoMem's retained budget. Panel (a) uses RULER Cohort A and full-set LoCoMo lexical scoring. CoMem reads its bounded pack from the full store without YaRN; the SnapKV/PyramidKV 64k/128k columns use YaRN factor 4 and see the full prompt. †Their native 15-cell macros include 64k/128k inputs left-truncated to the 40,960-token native window; within 8k–32k both score 99.96. Panel (b) times full-prompt prefill, compression, and 64-token decode (median of three); the final column is decode latency. These methods must prefill the full prompt before eviction. CoMem's separate L20A cohort peaks at 18.54 GB at 128k (Appendix A.4); hardware differs, so we report no cross-table latency ratio.

ple IDs, decode parameters, and per-example retrieved chunk IDs, order, and pack tokens; only replay start differs. Across 15 RULER Cohort-B cells (n=100 each), full replay scores 99.19 and layer-12 replay scores 96.07. The paired 3.12-point gap has bootstrap 95% CI $[2.36, 3.93]$ and exact McNemar p=8.79×10^{-24} (83 full-only correct versus one layer-12-only correct). The gap is largest on distractor-heavy multikey retrieval.

For latency, each arm uses the same approximately 6.5k-token pack retrieved from 16k source contexts in three independent processes, with three warmups and 20 reads per process. Layer-12 replay is $1.403\times$ faster (931.9 to 664.4 ms), with all three process-level ratios between 1.402 and 1.404. Total-decode medians remain similar at roughly 2.76–2.86 s. A preceding component harness attributes the read reduction primarily to upper-transformer forward ($1.398\times$), with unchanged LM-head time. Retrieval, index construction, reusable store/query Write, and persistent I/O are outside this read-phase timing. The result is therefore a measured quality–latency trade-off, not quality-preserving or end-to-end acceleration.

Replay start	RULER	Read (ms)	p10	p90
j=0	**99.19**	931.9	931.6	942.0
j=12	96.07	**664.4**	663.8	667.1
Trade-off	−3.12 pt	1.403× speedup		

Table 28: Paired quality–latency control with the same LoRA. Quality is the 15-cell RULER macro (n=1,500 paired examples); the 3.12-point loss has paired-bootstrap 95% CI $[2.36, 3.93]$. Latency uses a fixed approximately 6.5k-token pack retrieved from 16k source contexts and reports medians across three independent processes (20 reads each). Retrieval, reusable Write, I/O, and total decode are excluded; total decode is similar across arms.

A.5 Training-seed robustness

We trained two additional j=12 rank-32 LoRA adapters on the same data budget and evaluated all three seeds, including the flagship, on identical cells and sample counts (RULER n=50, BABILong n=100). Across the 18 reported RULER and BABILong cells, the median cell-wise standard deviation is 1.34 points and the maximum is 4.36; the flagship falls within the observed range on the RULER headline cells and is, if anything, slightly conservative on the two BABILong

Replay pathway	RULER B	MK 8/16k	Reusable
Full replay from j=0	99.19	100.0	No
Continuous-prefix h_{12} oracle	99.19	100.0	No
Chunk-local cached h_{12}	96.07	92.5	Yes

Table 29: Write-side attribution with identical examples, packs, and LoRA. The continuous-prefix oracle runs layers 0–11 over the selected packed sequence for every query, then starts upper replay from that h_{12}; it is bit-identical to full replay on all 1,500 Cohort-B examples. The deployable chunk-local path loses 3.12 points (95% CI $[2.36, 3.93]$; McNemar p=$8.79{\times}10^{-24}$), and 7.50 points on the pooled 8k/16k multikey cells (n=200, CI $[4.0, 11.5]$). The oracle is an attribution upper bound, not a reusable cache.

Backend	Write GB/s	Fetch ms	H2D ms	Peak QPS	16M outcome
GPU resident	OOM	–	–	–	128 GiB > 95 GiB
CPU pinned	139.6	0.81	1.41	941	235 GB host
Local NVMe	2.45	13.42	0.91	264	2.18 GB host
Network/CEPH	1.66	79.20	1.20	42	2.19 GB host

Table 30: Measured persistent-store tiers at 16M tokens (128 GiB of bf16 h_{12}). Each query fetches the same 50.3 MB top-12 pack; medians use seven repetitions after two warmups, and QPS is the best of 1/4/16 concurrent clients. GPU residence fits 8M tokens (64.98 GB peak) but not 16M; off-GPU tiers retain store-size-independent transfer volume at increasing I/O latency. This microbenchmark times storage and transfer, not BM25 or model inference.

cells that drive the maximum. The two added seeds use effective batch 3 rather than 8, so this remains a matched-data-budget robustness check with slightly different optimization noise rather than a fully controlled three-seed estimate.

A.6 Prompt and native-chat sensitivity

For Qwen3-8B, the GPT-4o LoCoMo judge is substantially less prompt-sensitive than token F1. CoMem scores 38.27 without chat templating and 37.76 with it; KV-Direct scores 34.59 and 38.22. In contrast, CoMem's F1/substring accuracy changes from 9.15/23.36 to 19.51/28.65, while KV-Direct F1 changes from 9.02 to 40.06, showing that lexical metrics are strongly formatting-sensitive. We therefore retain the uniform no-chat judge comparison as the controlled headline.

The official MemoryLLM chat template does not improve the tested BABILong, RULER, or LoCoMo diagnostics; full results are included in the released artifacts. We therefore keep the uniform no-chat row as the external persistent- memory reference and do not mix prompt protocols within an average.

Model	j	single	multikey	VT	macro
0.6B	2	99.80	85.48	62.47	85.68
1.7B	3	99.08	48.08	44.20	66.81
4B	6	96.36	35.48	42.50	60.52
14B	13	95.28	34.44	13.10	52.91
30B-A3B	12	99.44	57.56	87.07	80.48
32B	27	100.00	93.60	59.47	88.18†

Table 31: Completed adapter-free Qwen3-family RULER sweep. Values are means over five lengths for single and multikey (8k–128k), three for variable tracking (8k–32k), and all 13 cells for the macro. All cells use the same no-chat, chunk-512 protocol and n=500, except †32B (n=25). Scale is non-monotonic: architecture, selected split, and task family matter in addition to parameter count.

A.7 Cross-scale and MoE generality

Table 31 reports the completed no-chat adapter-free RULER sweep across six Qwen3 sizes. Performance is non-monotonic because the selected split, architecture, and task family vary with scale; we do not infer a scaling law. Table 32 evaluates a RULER-derived conservative integer split for each frozen backbone on all 1,986 LoCoMo questions. The no-chat score improves strongly overall (11.48→51.81) but is not strictly monotone. In particular, inspection of the 14B generations shows unusually long chain-of-thought trajectories without the chat template: reasoning consumes much of the generation budget before the final answer, plausibly explaining its dip below 8B. The chat template mitigates this behavior (28.80→39.58), while it slightly reduces 8B and 32B; we therefore treat the chat column as a generation-protocol sensitivity analysis rather than a second scaling curve. Table 33 adds Qwen3-30B-A3B, where Dense OOMs at 128k but CoMem runs at 63.0 GB and scores 100 on RULER single needle.

A.7.1 Hunyuan Hy3: an 80-layer sparse MoE

We additionally port CoMem to Tencent Hunyuan Hy3 (Tencent Hunyuan Team, 2026), an 80-layer sparse Mixture-of-Experts model (192 experts, top-8 routing plus one shared expert; ≈ 597 GB in bf16) with a 262,144-token native window. The backbone is sharded across eight GPUs; Write and Read semantics are unchanged.

Exact partition through MoE routing. Reconstructing a contiguous forward by writing `layers[0:j]` and resuming `layers[j:80]` gives $\max|\Delta\text{logit}| = 0.0$ at $j \in \{0, 1, 40, 80\}$,

Model	Safe j	j/L	No chat	Chat
Qwen3-0.6B	2	0.07	11.48	20.49
Qwen3-1.7B	3	0.11	17.42	26.44
Qwen3-4B	9	0.25	24.07	32.02
Qwen3-8B	9	0.25	32.38	31.52
Qwen3-14B	13	0.325	28.80	39.58
Qwen3-32B	27	0.42	**51.81**	**48.89**

Table 32: **Frozen CoMem transfers across the Qwen3 family on LoCoMo.** Each integer split is a conservative readout-safe point selected from RULER, independently of LoCoMo; it is not the 50% readout-crash boundary in Table 6. Values are full-set GPT-4o Judge accuracy (%) over all 1,986 questions. Both protocols use iterative BM25 (top-12, hop width 4), 512-token chunks, a BOS sink, greedy decoding, and a 512-token generation cap; the chat column is a template-sensitivity control. The 14B no-chat dip reflects unusually long chain-of-thought trajectories that delay the final answer; the chat template mitigates this behavior. Scores improve strongly with scale but are not strictly monotone.

Length	Prefill speedup	Dense mem	CoMem mem
8k	1.4×	63.7 GB	63.0 GB
32k	8.4×	71.3 GB	63.0 GB
128k	**Dense OOM**	OOM	**63.0 GB**

Table 33: CoMem on **Qwen3-30B-A3B** (48-layer sparse MoE; prefill speedup = Dense/CoMem prefill time). CoMem peak memory is context-independent (63.0 GB across 8k/32k/128k) while the full-context Dense reference grows and *OOMs* at 128k on an NVIDIA H20. At 128k CoMem still runs and scores RULER `niah_single` 100 (Dense OOM→CoMem 100); its prefill stays < 0.7 s at every length. The depth partition, retrieval, and constant read thus hold at sparse-MoE scale.

including the Dense-to-MoE boundary. The resumed half re-executes every router and expert, so the partition preserves routing exactly on this self-test.

Split depth and self-distillation. A PG19 sweep places the best readout region near $j \approx 32$–36. At $j = 32$, self-distillation reduces the 16k multiplicative LM tax from 1.496 to 1.078 and raises top-1 agreement from 0.773 to 0.871 (Table 34).

Bounded read through 256k. With the distilled adapter and BM25 top-8, the read stays at $\approx$ 4.3–4.6k tokens from 16k through 256k. CoMem reaches 100/98 single/multikey recall at the 256k native ceiling with no OOM (Table 35).

	ctx 8k		ctx 16k	
j=32	gap	top1	gap	top1
zero-shot	1.378	0.823	1.496	0.773
distilled	**1.071**	**0.895**	**1.078**	**0.871**

Table 34: CoMem self-distillation on Hy3 at split j=32 (LoRA r=32, α=64 on `layers[32:]`, teacher j=0 RAG recompute, 4000 steps of plain PG19 KL, no synthetic data). **gap** is the multiplicative LM tax (CoMem-readout perplexity / full-context perplexity; 1.0 is exact); **top1** is argmax agreement with the full forward, over matched 16-doc PG19 windows. The 16k LM tax falls from $+49.6\%$ to $+7.8\%$ and top1 rises $0.773 \rightarrow 0.871$, reproducing the 8B-backbone self-distillation effect on the 80-layer MoE.

Task	16k	32k	64k	128k	256k
niah_single_2	98	92	100	98	**100**
niah_multikey_1	100	94	100	100	**98**
read length (tok)	4.3k	4.6k	4.5k	4.4k	4.5k
context (tok)	16k	32k	65k	131k	261k

Table 35: CoMem on the Tencent Hunyuan **Hy3** backbone (`hy_v3`, 80-layer sparse MoE), RULER needle recall (n=50, official `string_match_all`), split j=32, BM25 top-8, self-distilled LoRA. The read length fed to the model stays ≈4.3–4.6k tokens while the context grows 16× (16k→256k); CoMem retains 100/98 recall at the backbone's 256k native ceiling, with 0 out-of-memory failures. The context row reports the mean packed length of the RULER samples.

B Chat-Template-Free Statistical Verification

All statistics in this appendix were recomputed from the saved prediction shards on the shared result filesystem, without GPU reruns. The Qwen3-8B flagship uses the self-distilled j=12 adapter, chunk 512, top-12 retrieval, a BOS sink, and no chat template. The released adapter artifact is identified by the SHA-256 and configuration in Table 25. The main RULER directory uses the `iter_bm25` implementation; the saved configuration records `rounds=0` and an observed read of approximately 6.5k tokens. Every reported RULER cell contains eight of eight shards, 100 examples, no empty prediction, and zero mismatch between the stored score and a fresh official-kernel recomputation. BABILong likewise contains 100 valid examples in every cell.

B.1 RULER cohort definitions

Cohort A is the all-method table (Appendix Table 16): it substitutes `niah_single_2`, keeps the same multikey/tracking tasks and n=100, and gives CoMem 97.05. Cohort B above was freshly sampled with `niah_single_3` because that is the exact task available for both YaRN arms and j=0; it gives 96.07. Tables mark every RULER aggregate with A or B, never subtract across them, and exclude legacy 256k cells. Table 20 contains only Cohort B.

Task	8k	16k	32k	64k	128k
niah_single_3	100.0	91.0	97.0	98.0	98.0
niah_multikey_1	94.0	91.0	99.0	90.0	93.0
variable_tracking	96.2	98.0	98.2	98.6	99.0

Table 36: RULER **Cohort B**, used for the matched length-extension and j=0 comparisons. The n=100 macro is $1441.0/15 = 96.0667$; the exact single-needle variant is `niah_single_3`.

B.2 LoCoMo judge protocol and denominator

Prediction shards for all methods and the adapter-free CoMem control deduplicate to all 1,986 LoCoMo items. GPT-4o judges the 1,540 answerable items in categories 1–4. Category 5 contains 446 adversarial abstention items and is scored locally: an answer is correct iff it is empty or matches the refusal rule. Thus the canonical full-set judge is over all 1,986 items, not only the API-judged subset.

Method	Cat. 1	Cat. 2	Cat. 3	Cat. 4	Cat. 5	Overall
CoMem + LoRA	**26.95**	**19.00**	**30.21**	**69.32**	2.47	**38.27**
CoMem frozen (j=9)	18.79	13.40	27.08	53.75	1.12	29.15
KV-Direct	24.11	18.69	25.00	62.19	2.69	34.59
StreamingLLM	22.70	11.21	23.96	42.21	6.95	25.63
InfLLM	18.44	14.33	25.00	33.89	**7.62**	22.21
MemoryLLM	15.60	8.72	15.63	27.47	0.45	16.11

Table 37: LoCoMo GPT-4o judge by category. Categories 1–4 contain 282/321/96/841 judged items; Category 5 contains 446 locally scored abstention items. Overall aggregates all 1,986 examples.

The judge uses an OpenAI-compatible chat-completions endpoint with model name `gpt-4o`, `seed=1`, and no client-set temperature or top-p. The endpoint does not expose a dated model snapshot. Requests are retried four times with exponential backoff; unparsable responses and API failures are conservatively scored wrong. The prompt presents the question, gold answer, and prediction; it accepts any semantic match (including equivalent date phrasing), rejects contradictions, omissions, empty answers, and refusals, and requires exactly `CORRECT` or `WRONG`. The verbatim template is released with the evaluation script. Gold alternatives are joined with "OR." A reply must begin with `CORRECT` or `WRONG`; a fallback substring vote is used before defaulting to wrong.

On the common GPT-4o-judged items (n=1540), CoMem scores 48.64 and KV-Direct 43.83. A paired per-item bootstrap of their difference (10,000 resamples, seed 1234)

gives $+4.81$ points with 95% CI $[2.34, 7.34]$. Because the questions are nested within only 10 conversations, this interval is not our sole inferential check. A paired conversation-cluster bootstrap, resampling the 10 conversations rather than individual questions (same 10,000 resamples, seed 1234), gives the same $+4.81$ point estimate with 95% CI $[1.27, 8.32]$ — wider than the per-item interval, as expected with only ten clusters; 8 of the 10 observed conversation-level differences favor CoMem. We do not interpret the bootstrap tail fraction as a calibrated hypothesis-test p value. We report the latter as a dependence-aware robustness check rather than claiming that 1,540 questions are independent. The canonical full-set point scores are 38.27 and 34.59, respectively.

B.3 Independent-judge audit

We reran the binary prompt with DeepSeek-V3 on a stratified 200-item subset shared by CoMem and KV-Direct. Agreement with GPT-4o is 0.81 ($\kappa = 0.626$). Both judges preserve the ordering: GPT-4o scores 36.5/32.5 ($+4.0$) and DeepSeek-V3 56.0/49.0 ($+7.0$) for CoMem/KV-Direct. Absolute calibration differs, but the positive gap is judge-robust.

B.4 Uncertainty and aggregation checks

Per-method, 1,000-resample item-bootstrap intervals (seed 1234) are $[36.20, 40.58]$ for CoMem's full-set LoCoMo judge, $[32.48, 36.71]$ for KV-Direct's, $[46.04, 50.98]$ and $[41.23, 46.30]$ for their category-1–4 judge scores, and $[8.60, 9.72]$ and $[8.51, 9.61]$ for their F1 scores. These unpaired, item-level intervals are descriptive; the paired cluster analysis above is the appropriate dependence-aware comparison.

We do not reuse cell-bootstrap intervals from the superseded RULER cohort or legacy BABILong predictions. The separate six-length LongEval aggregate is 72.83 over 600 examples (95% CI $[69.33, 76.33]$); Table 21 uses the predefined five-length 8k–128k mean of 69.0.